A PROJECT REPORT ON

Plant Leaf Disease Multilevel Classification using Few Shot Learning

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY,
PUNE IN THE PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE AWARD OF THE DEGREE

BACHELOR OF ENGINEERING
(Computer Engineering)

BY

Aishwarya Behere	B400310398
Rhutam Mahajan	B400310454
Ashutosh Sabnis	B400310491
Vaishnavi Salunkhe	B400310521

Under the guidance of

Prof. Dr. Mrs. S. A. Itkar

DEPARTMENT OF COMPUTER ENGINEERING
PES MODERN COLLEGE OF ENGINEERING
SHIVAJINAGAR,PUNE 411005

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
2024 - 25

CERTIFICATE

This is to certify that the Project Entitled

Plant Leaf Disease Multilevel Classification using few shot learning

Submitted by

Aishwarya Behere	B400310398
Rhutam Mahajan	B400310454
Ashutosh Sabnis	B400310491
Vaishnavi Salunke	B400310521

is a bonafide work carried out by them under the supervision of Prof. Dr. Mrs. S. A. Itkar and it is approved for the partial fulfillment of the requirement of Savtribai Phule Pune university, Pune for the award of the degree of Bachelor of Engineering (Computer Engineering).

Prof. Dr. Mrs. S. A. Itkar
Guide
Department of Computer Engineering

Prof. Dr. Mrs. S. A. Itkar
Head
Department of Computer Engineering

Signature of Internal Examiner

Signature of External Examiner

A Project Report Titled as

Plant Leaf Disease Multilevel Classification Using Few-Shot Learning

is successfully completed by

Aishwarya Behere	B400310398
Rhutam Mahajan	B400310454
Ashutosh Sabnis	B400310491
Vaishnavi Salunke	B400310521

at

DEPARTMENT OF COMPUTER ENGINEERING

PES MODERN COLLEGE OF ENGINEERING

SAVITRIBAI PHULE PUNE UNIVERSITY,PUNE

ACADEMIC YEAR 2024-25

Prof. Dr. Mrs. S. A. Itkar
Guide
Department of Computer Engineering

Prof.Dr. Mrs. S. A. Itkar
Head
Department of Computer Engineering

Acknowledgement

It gives us pleasure in presenting the project report on **'Plant Leaf Disease Multilevel Classification using few shot Learning'**.

Firstly, we would like to express our indebtedness appreciation to our guide **Prof. Dr. Mrs. S. A. Itkar**.Her constant guidance and advice played very important role in successful completion of the project.She always gave us her suggestions, that were crucial in making this report as flawless as possible.

We would like to express our gratitude towards **Prof. Dr. Mrs. S. A. Itkar** Head of Computer Engineering Department, PES Modern College of Engineering for her kind co-operation and encouragement which helped us during the completion of this report.

Also we wish to thank our Principal, **Prof. Dr. Mrs. K. R. Joshi** and all faculty members for their whole hearted co-operation for completion of this report. We also thank our laboratory assistants for their valuable help in laboratory.

Last but not the least, the backbone of our success and confidence lies solely on blessings of dear parents and lovely friends.

Aishwarya Behere
Rhutam Mahajan
Ashutosh Sabnis
Vaishnavi Salunke

Abstract

The project focuses on the multilevel classification of plant leaf diseases using few-shot learning techniques, enabling accurate disease identification even with very limited training samples. The core of the system utilizes the EfficientNet-B0 model as a feature extractor due to its lightweight architecture and high accuracy with fewer parameters. This backbone extracts rich visual representations from plant leaf images, which are then passed to a Prototypical Network that performs classification based on distance metrics in the embedding space. By relying on few-shot learning, the system is capable of recognizing new disease classes with only a few labeled examples, making it highly suitable for real-world scenarios where annotated data is scarce.

The system follows a modular and scalable design, dividing the classification pipeline into key services such as image preprocessing, feature extraction using EfficientNet-B0, prototype computation, and prediction via the Prototypical Network. This separation of tasks allows for easy enhancement and tuning of individual components. The model is trained in an episodic manner, where support and query sets are dynamically generated during each training episode. This strategy helps the network to generalize across varying plant species and disease categories effectively, even under low-data conditions.

To maintain efficiency and simplicity, the architecture avoids heavy ensemble or transformer-based models, focusing instead on a streamlined few-shot approach. During inference, the system takes an input image and compares it to class prototypes formed from the support set to determine the most probable disease class. The overall framework is designed for deployment in practical agricultural settings, providing a lightweight and fast solution that can be accessed via a simple user interface. Farmers and agricultural workers can upload leaf images and receive instant disease predictions, enabling early detection and effective crop management with minimal technical overhead.

Keywords

Few-Shot Learning, EfficientNet-B0, Prototypical Networks, Plant Leaf Disease Detection, Image Classification, Low-Data Learning, Feature Embedding, Agricultural AI.

Contents

1 Introduction **1**
1.1 Motivation . . . 2
1.2 Problem Definition and Objectives . . . 2
1.2.1 Problem Definition . . . 2
1.2.2 Objectives . . . 2
1.3 Project Scope . . . 2

2 Literature survey **3**

3 Software Requirements Specification **5**
3.1 Assumptions and Dependencies . . . 6
3.2 Functional Requirements . . . 6
3.2.1 Hardware Requirements . . . 6
3.3 External Interface Requirements (if any) . . . 6
3.3.1 Hardware Interfaces . . . 6
3.3.2 Communication Interfaces . . . 6
3.4 Nonfunctional Requirements . . . 7
3.4.1 Performance Requirements . . . 7
3.4.2 Safety Requirements . . . 7
3.4.3 Security Requirements . . . 7
3.4.4 Software Quality Attributes . . . 7
3.5 System Requirements . . . 7
3.5.1 Database Requirements . . . 7
3.5.2 Software Requirements(Platform Choice) . . . 7
3.5.3 Hardware Requirements . . . 7
3.6 Analysis Models:SDLC Model to be applied . . . 7

4 System Design **8**
4.1 System Architecture . . . 9
4.1.1 Data Collection . . . 9
4.1.2 Image Acquisition . . . 9
4.1.3 Dataset Cleaning . . . 9
4.1.4 Few Shot Split Creation at a set Cleaning . . . 9
4.1.5 Preprocessing and Augmentation . . . 9
4.1.6 Model Evaluation . . . 9
4.1.7 Hierarchical Protoypical Network Training . . . 10
4.1.8 Algorithm . . . 10
4.2 Mathematical Model . . . 11
4.3 Data Flow Diagrams / UML Diagrams . . . 13
4.3.1 Data Flow Diagram . . . 13
4.3.2 Class Diagram . . . 14

4.3.3 Sequence Diagram . . . 15
4.3.4 Activity Diagram . . . 16
4.3.5 Use Case Diagram . . . 17
4.3.6 Entity Relationship Diagram . . . 18
4.3.7 Deployment Diagram . . . 19

5 Project Plan **20**
5.1 Project Estimate . . . 21
5.1.1 Reconciled Estimates . . . 21
5.1.2 Human Resources . . . 21
5.1.3 Software And Hardware Requirements . . . 21
5.2 Risk Management . . . 21
5.2.1 Risk Identification . . . 21
5.2.2 Risk Analysis . . . 21
5.2.3 Overview of Risk Mitigation, Monitoring, Management . . . 22
5.3 Project Schedule . . . 22
5.3.1 Project Task Set . . . 22
5.3.2 Task Network . . . 22
5.4 Team Organization and Timeline Chart . . . 23
5.4.1 Module Description . . . 24

6 Project Implementation **25**
6.1 Overview of Project Modules . . . 26
6.2 Tools and Technologies Used . . . 26
6.3 Algorithm Details . . . 26

7 Software Testing **28**
7.1 Types of Testing . . . 29
7.2 Test cases . . . 30

8 Results **32**
8.1 Outcomes . . . 33
8.1.1 Functional Accuracy . . . 33
8.2 Testing Results . . . 33
8.3 Observations . . . 33
8.4 Interface Snapshots . . . 34

9 Conclusions **38**
9.1 Conclusions . . . 39
9.2 Future Scope . . . 39
9.3 Applications . . . 39
9.4 Limitations . . . 39

Annexure A **40**

Annexure B **42**

Annexure C **44**

10 References **46**

List of Figures

4.1 System Architecture 9
4.2 Data Flow Diagram 13
4.3 Class Diagram 14
4.4 Sequence Diagram 15
4.5 Activity Diagram 16
4.6 Use Case Diagram 17
4.7 Entity Relationship Diagram 18
4.8 Deployment Diagram 19

8.1 Home Page Interface 34
8.2 User Login Page 35
8.3 Dashboard 36
8.4 Prediction Output Page 37

C.1 Plagiarism Report 45

List of Tables

2.1 Literature Survey 4

5.1 Timeline chart 23
5.2 Team Members 23

7.1 Functional Testing Test Cases 30
7.2 Integration Testing Test Cases 30
7.3 System Testing Test Cases 31

8.1 System Testing Summary 33

CHAPTER 1
INTRODUCTION

1.1 Motivation

With the increasing global demand for sustainable agriculture and the growing threat of plant diseases, effective monitoring tools are essential for ensuring food security. However, traditional disease diagnosis methods rely heavily on large annotated datasets and often struggle to generalize to new disease classes with scarce samples. This project employs a few-shot learning framework combining EfficientNet-B0 for robust feature extraction and Prototypical Networks for classification, enabling rapid and accurate multilevel plant leaf disease detection with minimal labeled examples.

1.2 Problem Definition and Objectives

1.2.1 Problem Definition

To develop an automated multilevel plant leaf disease classification system using few-shot learning—leveraging EfficientNet-B0 for feature extraction and Prototypical Networks for classification—to achieve accurate diagnosis from minimal labelled examples.

1.2.2 Objectives

- To analyze and study existing approaches for multilevel plant leaf disease classification using few-shot learning techniques
- To collect and preprocess a suitable dataset of plant leaf images, including various disease categories.
- To implement a deep learning-based model using EfficientNet-B0 for feature extraction and Prototypical Networks for few-shot disease classification.

1.3 Project Scope

This project focuses on building a deep learning-based system for multilevel plant leaf disease classification using few-shot learning. It involves dataset collection, model training using EfficientNet-B0 for feature extraction, and Prototypical Networks for classification. The system is limited to disease classification and does not include real-time deployment, hardware optimization, or additional features such as environmental factors analysis.

CHAPTER 2

LITERATURE SURVEY

Title	Author	Year	Publication	Description
Plant Leaf Disease Detection, Classification, and Diagnosis Using Computer Vision and Artificial Intelligence: A Review	Anuja Bhargava, Aashesh shukla, Prakash Goswami, Mohammed H Alsharif, Peerapong Uthansakul and Monthippa Uthansakul	2024	IEEE	Deep learning, diagnosis, image processing, machine learning, plant disease.
Plant Disease Detection and Classification by Deep Learning—A Review	Lili Li,Shujuan Zhang AND Bing Wang	2021	IEEE	The paper utilizes deep learning techniques, primarily Convolutional Neural Networks (CNNs), for automated detection and classification of plant diseases from leaf images.
Plant Leaf Diseases Classification Based on Few-Shot Learning	Wei Wu	2023	Academic Journal of Science and Technology	This paper employs few-shot learning techniques to classify plant leaf diseases with minimal labeled training data.
Machine Learning and Deep Learning for Plant Disease Classification and Detection	Vasilos Balafas , Emmanouil Karantoumanis , Malamati Louta AND Nikolaos Ploskas.	2023	IEEE	This paper employs both machine learning algorithms (like SVM, KNN) and deep learning models (such as CNNs) to classify and detect plant diseases from image datasets.

Table 2.1: Literature Survey

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

- Input leaf images are expected to be of high resolution (224 × 224 px), captured under consistent lighting with minimal background clutter or occlusion, so that the EfficientNet-B0 backbone can reliably extract discriminative features; images with severe blur, shadows, or color casts may degrade classification performance.
- The system depends on a modern deep-learning framework (PyTorch or TensorFlow), uses EfficientNet-B0 pretrained on ImageNet for transfer learning, and implements a Prototypical Network in PyTorch Lightning (or equivalent) for few-shot classification; additional libraries include OpenCV and Albumentations for preprocessing/augmentation, Scikit-learn for metrics, and FastAPI.
- A well-curated, labeled dataset is assumed, providing episodic support and query sets with at least K examples per disease class to enable meta-training; a GPU-enabled environment (CUDA-compatible) is required for efficient backpropagation during training, while inference should remain feasible on standard CPUs or edge devices with the necessary runtime dependencies.

3.2 Functional Requirements

3.2.1 Hardware Requirements

- Deep learning models, CNN, involve heavy computations due to their need to process spatiotemporal data.
- Minimum of 8 GB of RAM, though 16 GB or more is recommended for handling large image datasets efficiently.
- 1 TB of SSD storage for fast read/write speeds during training.
- At least 850W or higher, depending on the hardware configuration.

3.3 External Interface Requirements (if any)

3.3.1 Hardware Interfaces

Support for uploading high-resolution plant leaf images through camera interfaces or image files. The system is compatible with common image formats such as JPEG, PNG, and TIFF for input, ensuring that images are processed at high resolution ($\geq 224 \times 224$ px) for accurate feature extraction.

3.3.2 Communication Interfaces

The system includes a notification service that can send real-time disease classification results and alerts through email or SMS.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

The system must be able to process plant leaf images in real-time, ensuring that classification and prediction tasks are completed within 2-3 seconds per image, without compromising accuracy.

3.4.2 Safety Requirements

All plant leaf image data is handled in compliance with relevant agricultural data privacy and safety laws. The system ensures that any data storage or transmission is secure and protects sensitive information regarding plant health.

3.4.3 Security Requirements

The system guarantees that all images and classification results are securely transmitted and stored using SSL/TLS encryption, preventing unauthorized access and ensuring data confidentiality throughout the process.

3.4.4 Software Quality Attributes

The system should guarantee maximum uptime, ensuring continuous operation without frequent outages or failures. Redundancy should be implemented for critical components like the feature extraction and classification modules to avoid disruptions in service.

3.5 System Requirements

3.5.1 Database Requirements

The system should support the storage of large volumes of plant leaf image data, metadata, classification results, and logs. The database must scale to handle data from multiple users and instances, potentially utilizing sharding for efficient data management.

3.5.2 Software Requirements(Platform Choice)

Operating System, Windows 10, Deep Learning Frameworks, Programming Languages.

3.5.3 Hardware Requirements

Processor(CPU), Graphical Processing Unit(GPU), Memory(RAM), Storage, Network Bandwidth, Power Supply And Backup.

3.6 Analysis Models:SDLC Model to be applied

The Agile SDLC model is best suited for this project, as it allows for iterative development, testing, and enhancement of the deep learning model.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

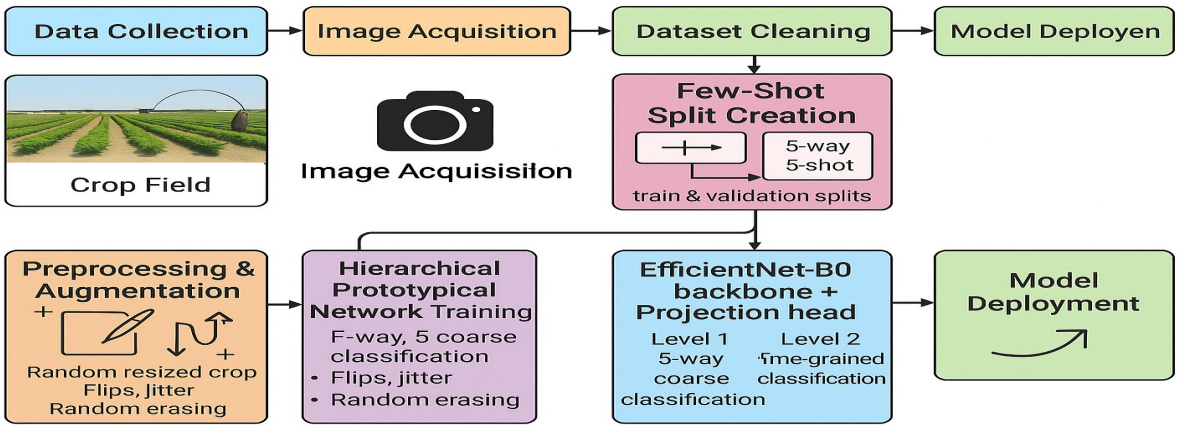

Figure 4.1: System Architecture

4.1.1 Data Collection

- Images of healthy and diseased leaves are gathered directly from crop fields, ensuring a diverse representation of plant species, disease stages, lighting conditions, and backgrounds.

4.1.2 Image Acquisition

- High-resolution photographs are captured using smartphones or DSLR cameras, then transferred to the processing pipeline in JPEG/PNG format.

4.1.3 Dataset Cleaning

4.1.4 Few Shot Split Creation at a set Cleaning

- The cleaned dataset is partitioned into episodic tasks: each episode contains a 5-way support set (5 examples per class) and a corresponding query set, enabling the Prototypical Network to learn distance-based classification from minimal samples.

4.1.5 Preprocessing and Augmentation

- Each image is resized and normalized, then subjected to random resized crops, horizontal/vertical flips, color jitter, and random erasing, increasing robustness to scale, orientation, and occlusion variations.

4.1.6 Model Evaluation

- Anomaly Score: A float between 0 and 1 indicating how likely an image or segment belongs to an anomalous class, used to evaluate the classifica-

tion of plant diseases Evaluation using AUC and PR curves: Measures the overall ability of the model to discriminate between diseased and healthy plant leaves, providing insights into how well the model differentiates between classes.

4.1.7 Hierarchical Protoypical Network Training

- Level 1 (Coarse Classification): the network first solves an F-way task e.g. healthy vs. disease.
- Level 2 (Fine-Grained Classification): using embeddings from Level 1, the network then performs a 5-way task across specific disease classes, refining its discriminative ability. RESTful FastAPI server exposes the model for external applications. It receives video input, processes it, and returns anomaly scores in real-time.

4.1.8 Algorithm

- Feature Extraction:
  - EfficientNet-B0 is used to extract high-quality embeddings from input leaf images, capturing detailed texture and shape patterns.
- Prototype Generation:
  - For each class in the support set, the mean embedding is computed to form class.
- Distance Calculation:
  - Each query image is compared to all prototypes using Euclidean distance in the embedding space.
- Classification:
  - The query image is assigned to the class whose prototype is closest, enabling accurate prediction with very few labelled samples.
- Training:
  - The model is trained in episodes (5-way 5-shot), where each episode mimics a few-shot task, allowing the network to generalize to new disease classes with limited data.

4.2 Mathematical Model

- The system $\mathcal{S}$ is mathematically defined as:

$$\mathcal{S} = \{I, P, O\}$$

where:

  - I = Input leaf images collected from few-shot disease datasets,
  - P = Set of processes including:
    * Image Preprocessing and augmentation
    * Feature extraction using EfficientNet-B0,
    * Prototype computation using class-wise support embeddings,
    * Classification using distance-based comparison,
  - O = Output label predicting the disease class of a query image.

The overall system flow can be described as:

$$\begin{aligned} V_i &\rightarrow \text{Feature Extraction}\,(\phi(I)) \\ &\rightarrow \text{Prototype Formation} \\ &\rightarrow \text{Distance Calculation} \\ &\rightarrow \text{Prediction}\ (argmin D(Pc, Q)) \\ &\rightarrow \text{Output (Disease Class)} \end{aligned}$$

where:

  - I_i is the ith input image,
  - $\phi(I)$ denotes the embedding extracted from image I using EfficientNet B0,
  - P_c is the Prototype vector for class c
  - Q is the embedding of the query image.
  - D(P_c, Q) is the Euclidean distance between the query image and each prototype.
  - The prediction is the class c for which this distance is minimum.

- **Feature Extraction:**

$$F = \phi(C)$$

where F is the feature vector for image I extracted using EfficientNet B0.

Distance Based Classification:

$$D(P_c, Q) = \|\phi(Q) - P_c\|^2$$

The system assigns the query image Q to the class c for which this distance is smallest.p is the probability score output using a Sigmoid activation function.

Decision Rule:

$$\text{Predicted Class} = \arg\min_{p \geq \theta} D(P_c, Q)$$

$$\text{Output} = \begin{cases} \text{Normal}, & \text{if } p < \theta \\ \arg\min_{p \geq \theta} D(P_c, Q), & \text{otherwise} \end{cases}$$

Loss Function for Training (MIL Loss):

$$L_{\text{MIL}} = \frac{1}{N} \sum_{i=1}^{N} [\max(0, 1 - \max(p_{\text{anomaly}}) + \max(p_{\text{normal}})) + \lambda(\text{sparsity} + \text{smoothness})]$$

where λ controls regularization terms to improve model stability and performance.

4.3 Data Flow Diagrams / UML Diagrams

4.3.1 Data Flow Diagram

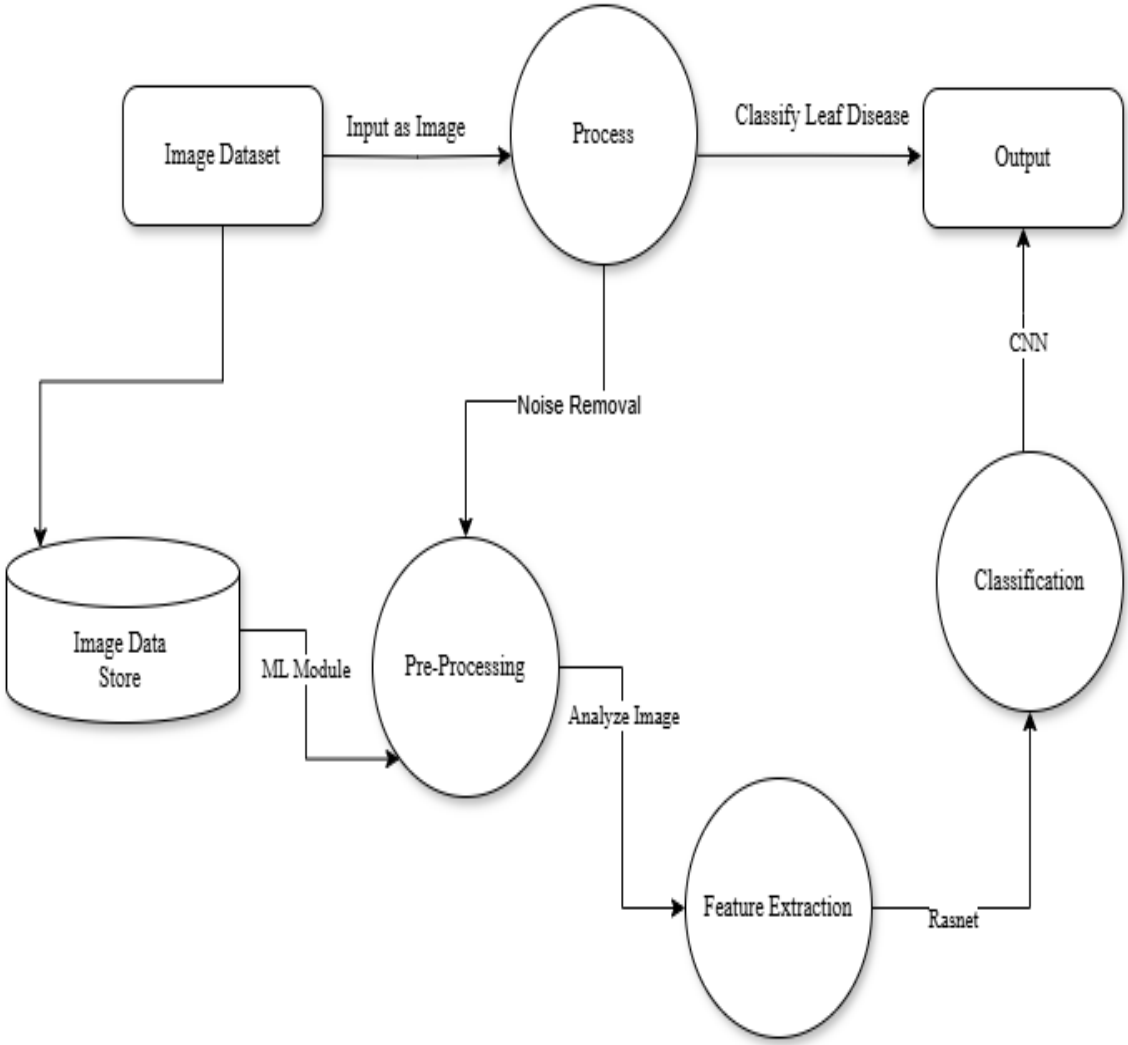

Figure 4.2: Data Flow Diagram

4.3.2 Class Diagram

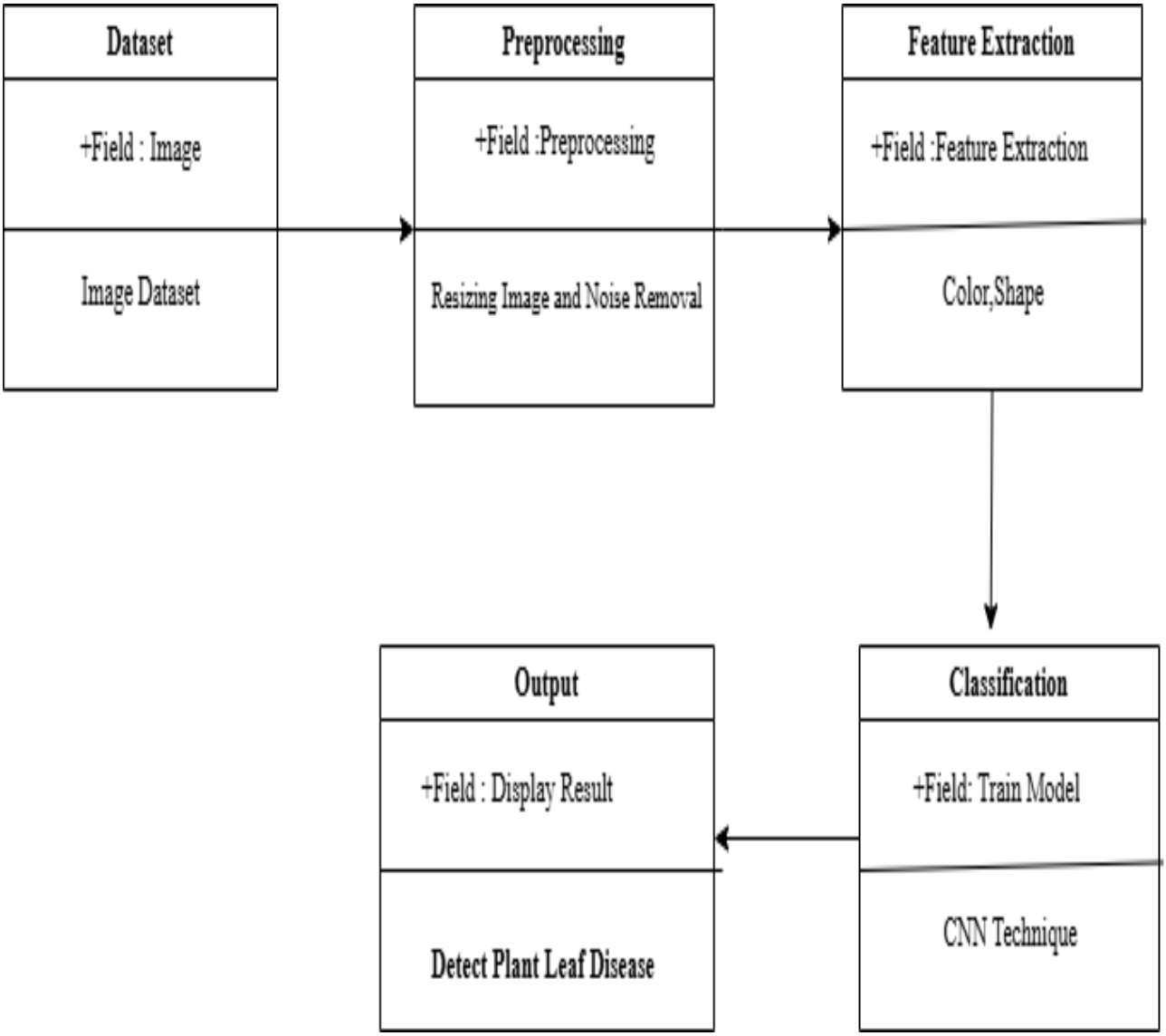

Figure 4.3: Class Diagram

4.3.3 Sequence Diagram

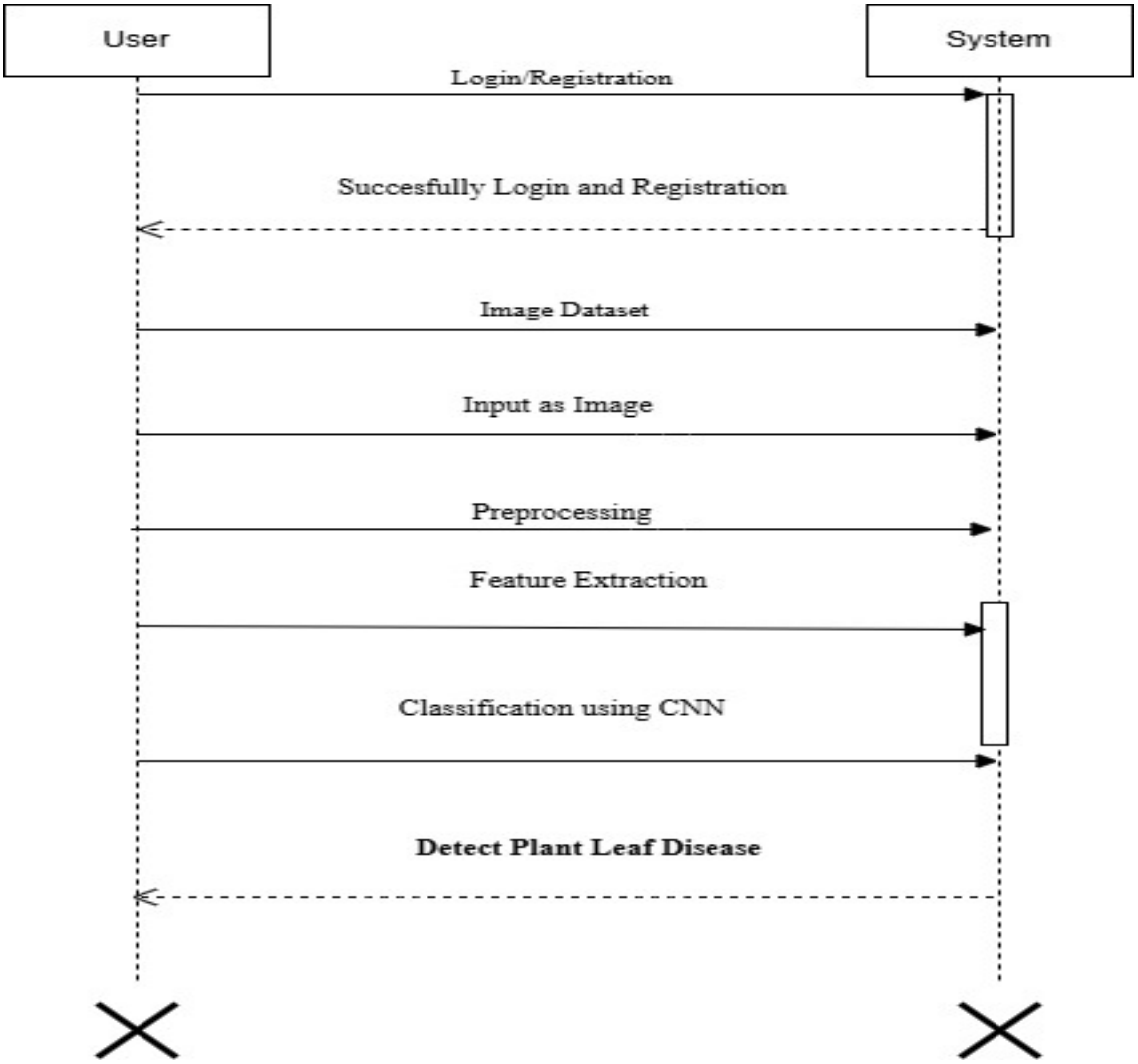

Figure 4.4: Sequence Diagram

4.3.4 Activity Diagram

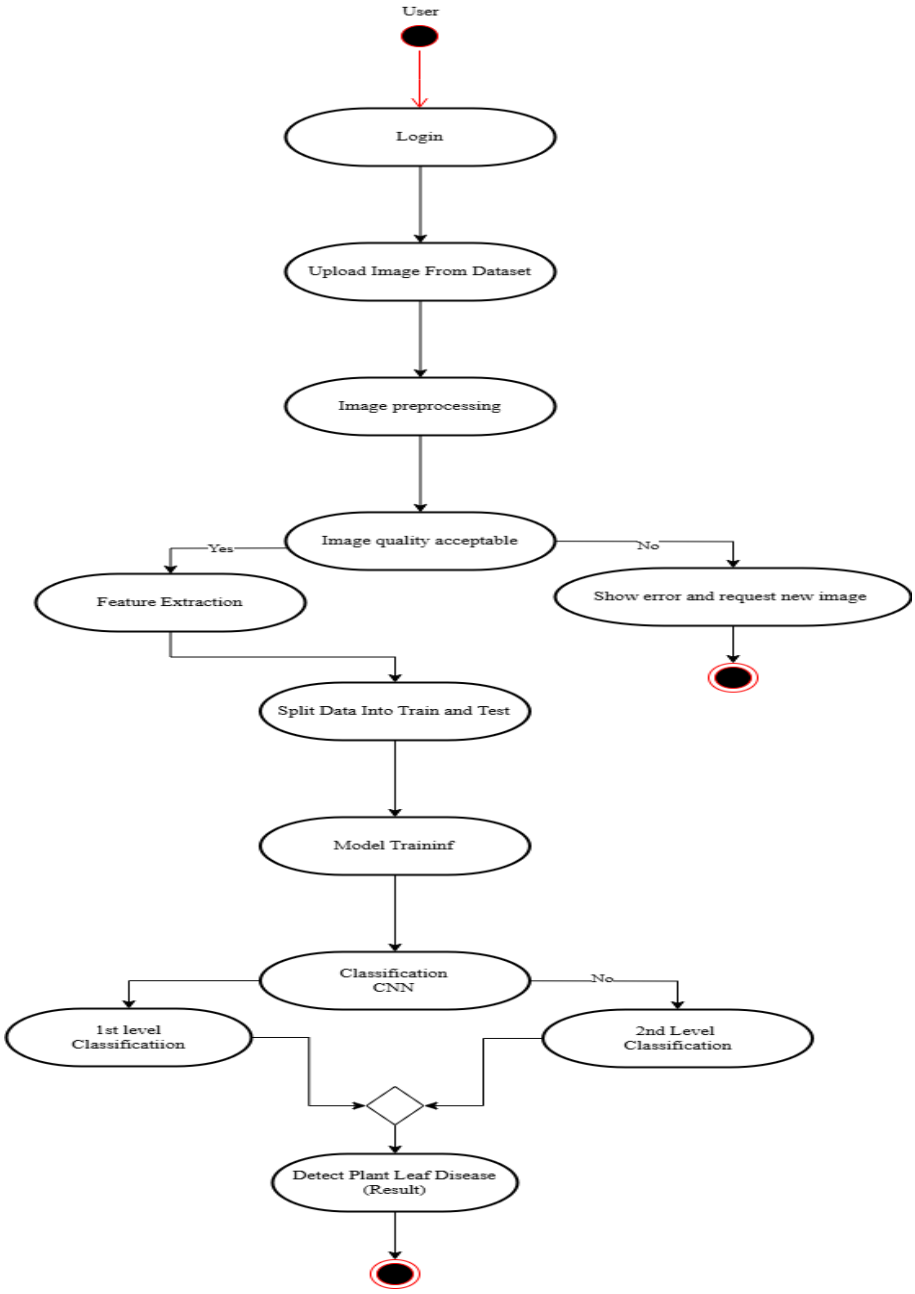

Figure 4.5: Activity Diagram

4.3.5 Use Case Diagram

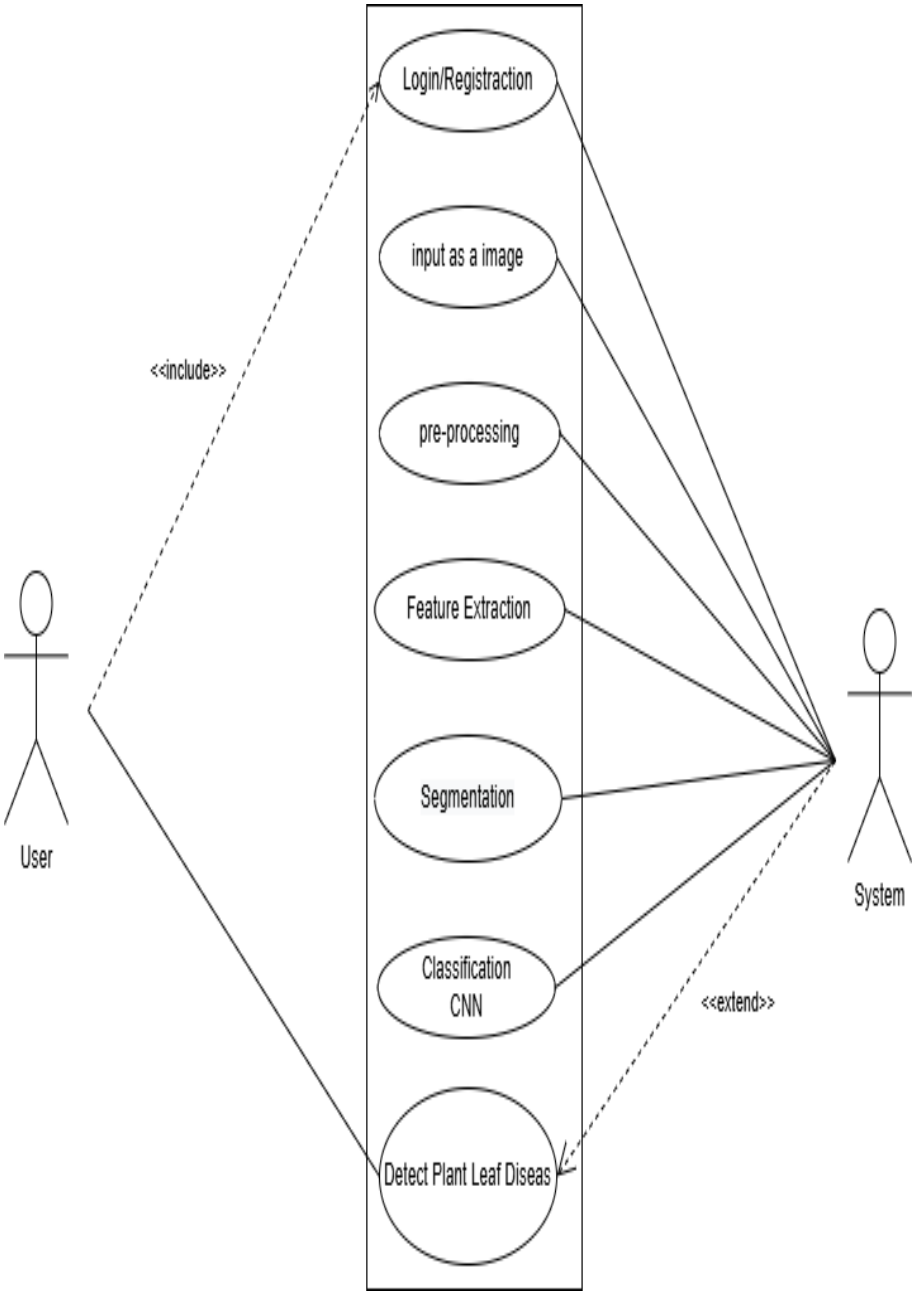

Figure 4.6: Use Case Diagram

4.3.6 Entity Relationship Diagram

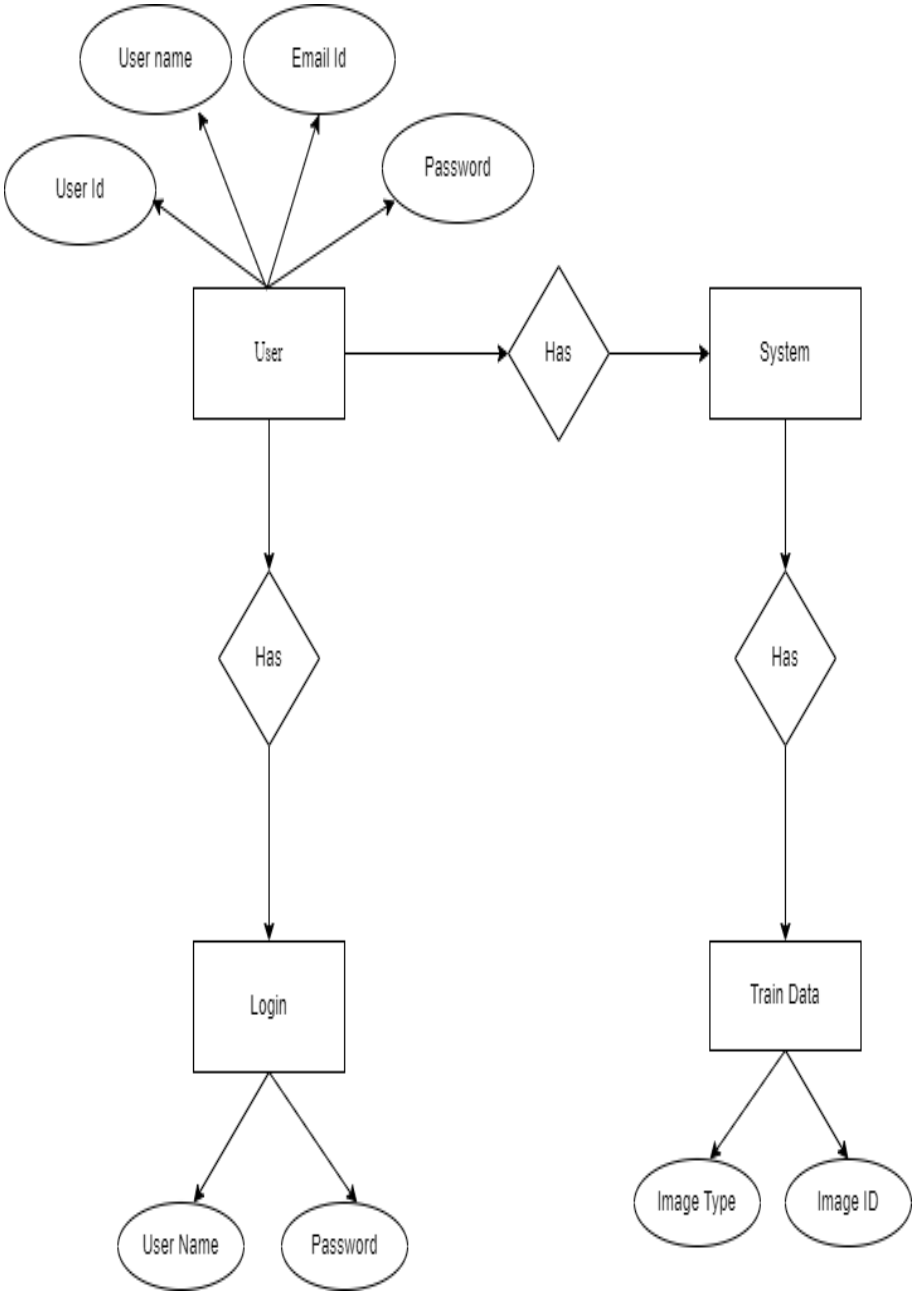

Figure 4.7: Entity Relationship Diagram

4.3.7 Deployment Diagram

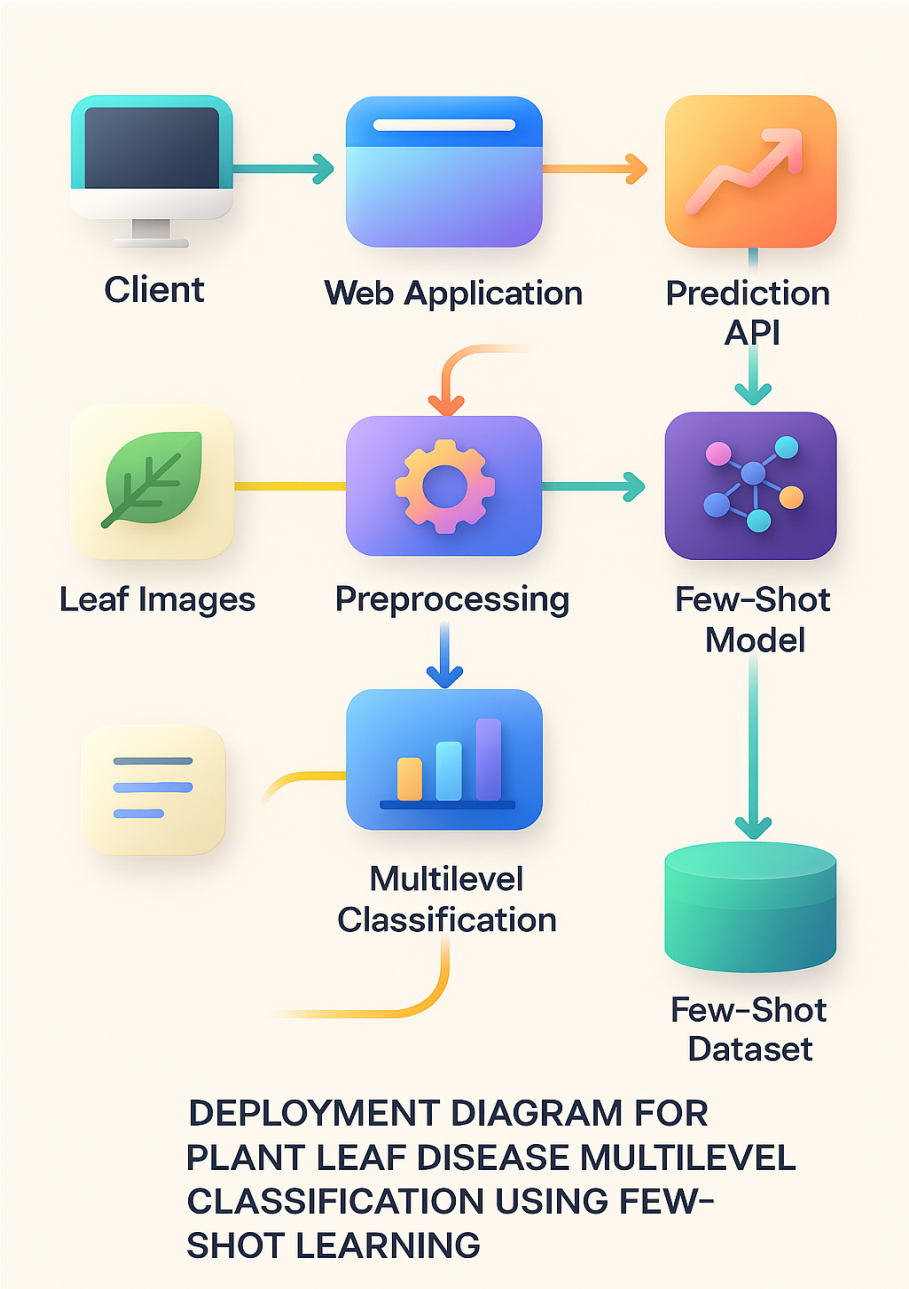

Figure 4.8: Deployment Diagram

CHAPTER 5
PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- The project was estimated by evaluating the efforts required for each phase: research, dataset preparation, model development, testing, and documentation. Based on function point analysis and expert judgment, the project required approximately 140-160 person-hours, distributed over 12-14 weeks. The initial high-level estimate was later refined after breaking down the tasks into smaller modules and reviewing the feasibility of integrating few shot learning for disease classification. .

5.1.2 Human Resources

- Deep Learning Model Developer (Team Lead)
- Backend Developer (for integration and testing)
- Research and Documentation Contributor

5.1.3 Software And Hardware Requirements

- Python, TensorFlow/Keras, OpenCV
- Jupyter Notebook
- VS Code for development
- Hardware: Laptop with minimum 8GB RAM and GPU support (preferably NVIDIA)

5.2 Risk Management

5.2.1 Risk Identification

1. Insufficient Dataset Quality
2. High Training Time and GPU Dependency
3. Model Underfitting
4. Few shot Learning Compatibility
5. Integration Delays

5.2.2 Risk Analysis

1. Dataset Quality Issues: High Priority due to potentially severe impact on model accuracy and reliability.
2. GPU/Training Time: High Priority to manage resource constraints and maintain project timelines.

3. Underfitting: Medium Priority as it can significantly harm model's ability to learn complex patterns.
4. Few Shot Learning Failures: Medium Priority due to the potential for increased development time if it fails.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

1. Dataset Quality: Use multiple datasets and perform extensive data preprocessing.
2. Underfitting: Use techniques like increasing model complexity and training for more epochs.
3. Model Selection: Evaluate multiple pre-trained model EfficientNet B0 for compatibility.
4. Integration Delays: Adopt an agile development cycle with weekly testing and module-wise integration.

5.3 Project Schedule

5.3.1 Project Task Set

1. Literature Review: 1 week to study existing systems and models.
2. Dataset Collection And Preprocessing: 2 weeks to gather, clean, and prepare the dataset.
3. Model Selection And Transfer Learning: 2 weeks to choose and adapt a pre-trained model.
4. Model Training And Validation: 3 weeks to train the model and validate its performance.
5. Backend Integration: 2 weeks to connect the model with the interface/API.
6. Testing And Evaluation: 2 weeks for functional and performance testing.
7. Report Writing And Finalization: 2 weeks to thoroughly document the project.

5.3.2 Task Network

Major task in project are:

- **Task 1** - Group formation
- **Task 2** - Deciding domain of interest
- **Task 3** - Topic finalization and approval
- **Task 4** - Literature survey

- **Task 5** - Information Gathering
- **Task 6** - Implementation I
- **Task 7** - Review
- **Task 8** - Report

5.4 Team Organization and Timeline Chart

No.	Task	Duration (Days)
1.	Group Formation	5
2.	Deciding area of interest	10
3.	Topic finalization and approval	15
4.	Literature survey	13
5.	Information gathering	20
6.	Implementation	60
7.	Review	10
8.	Report	5

Table 5.1: Timeline chart

Name	Task
Aishwarya Behere	Project Planning, Implementation and Review
Rhutam Mahajan	Project Planning, Implementation and Review
Ashutosh Sabnis	Project Planning, Implementation and Review
Vaishnavi Salunke	Project Planning, Implementation and Review
Prof.Dr.Mrs.S.A.Itkar	Guide

Table 5.2: Team Members

5.4.1 Module Description

1. Feature Extractor and Training Module :
   - Uses EfficientNet B0 to extract image embeddings from plant leaf images (resized and normalized).
   - Supports both pretrained inference and fine-tuning using Prototypical Networks for few-shot learning.
2. Few Shot Training Module
   - Employs episodic training with support and query sets to simulate few-shot tasks.
   - Uses prototypical loss for classification based on distances to class prototypes.
   - Optimization includes dropout, ReLU, and Adam optimizer.
3. Disease Recognition and Optimization Module :
   - Computes distances between input embeddings and class prototypes to classify leaf diseases.
   - Allows optional fine-tuning and includes MIL loss and basic regularization technique.
4. Evaluation and API Module:
   - Evaluate using accuracy, confusion matrix, precision score, recall score, F1 score.
   - FastAPI-based endpoints allow image uploads and return plant type and disease.
   - Supports thresholding for uncertain predictions
5. Communication and Backend Module :
   - Backend manages image storage, processing, classification, and response generation.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

6.2 Tools and Technologies Used

- Frameworks and Libraries :
  1. PyTorch – for designing and training deep learning models.
  2. NumPy, Pandas – for data manipulation and analysis
  3. Matplotlib, Seaborn – for result plotting and visualizations
- Models and development Environment :
  1. CNN – EfficientNet B0 and Visual Studio Code, Flask.

6.3 Algorithm Details

- A prototypical Network learns a metrics space in which classification is performed by computing distances to class prototypes. At inference, a small support set defines each class, and a query set is classified by the nearest prototype.

1. Data Preparation
   - Images from PlantVillage organized as folders: *PlantName_DiseaseName*.
   - For each class folder, extract embeddings via EfficientNet-B0:
     - Resize and center-crop to 224×224; normalize with ImageNet mean and standard deviation.
     - Remove the final classifier and L_2-normalize the 1,280-dimensional feature vector.
   - Compute average intra-class cosine similarity.
   - Discard classes where the average similarity is less than 0.7.
   - For each remaining class, randomly sample:
     - Support set: 10 images per class.
     - Query set: 10 images per class.
   - Construct separate `train` and `val` splits under `D:/fewshot_dataset`.
2. Model Architecture
   - Backbone: EfficientNet-B0 pretrained on ImageNet; classifier replaced by identity.
   - Embedding Head: 1,280 $\rightarrow$ 768 (ReLU, Dropout 0.7) $\rightarrow$ 384 (BatchNorm).
   - Classification Head: 384 $\rightarrow$ 256 (ReLU) $\rightarrow$ N classes.
   - L_2-normalize embeddings before feeding them to the head.
3. Episodic Training Procedure
   - Sample an Episode

  - Choose $N = 5$ classes at random.
  - For each class, sample $k = 10$ support and $q = 10$ query images.
- Prototype Computation
  - For each class c:
$$\mathbf{p}_{(c)} = \frac{1}{k} \sum_i \mathbf{e}_{(c,i)},$$
    where $\mathbf{e}_{(c,i)}$ are L_2-normalized support embeddings.
- Query Classification
  - Compute pairwise Euclidean distances between query embeddings and prototypes.
  - Convert to logits: $\ell_{j,c} = -\tau \cdot d(\mathbf{e}_q^{(j)}, \mathbf{p}_c)$, with $\tau = 20$.
  - Apply cross-entropy loss with label smoothing $\varepsilon = 0.2$.
- Metrics
  - Episode accuracy: Fraction of queries correctly classified.

4. Optimization
   - Optimizer: AdamW, $lr = 5 \times 10^{-5}$, weight decay $= 5 \times 10^{-4}$.
   - Scheduler: `ReduceLROnPlateau(monitor=val_avg_acc, factor=0.5, patience=3)`.
   - Early stopping:
     - No improvement over 5 epochs, or
     - Overfitting detected if train–val gap $> 5\%$ for 3 consecutive epochs.
5. Evaluation
   - Few-Shot (Validation) Evaluation: Evaluate on 10 validation episodes per epoch.
   - Full-Dataset Evaluation:
     - Treat trained model as a standard classifier on:
       * Full clean dataset: `D:/cleaned_plantvillage`
     - Compute:
       * Accuracy: $\left(\frac{\#\text{correct}}{\text{total}}\right) \times 100$
       * Precision, Recall, F1-score
     - Example Results:
       * Full dataset: Accuracy 97.77%, Precision 95.46%, Recall 96.40%, F1 95.83%

CHAPTER 7

SOFTWARE TESTING

7.1 Types of Testing

1. Model Testing :
   - Validation Testing:
     - Evaluates performance on the validation set during training.
     - Helps in tuning hyperparameters like learning rate, number of epochs, etc.
     - Metrics used: Accuracy, Loss, possibly Precision, Recall, and F1-score
   - Evaluation Testing
     - Performed on the test set or query set after training.
     - Tests generalization capability of the model on unseen data.
     - Measures final performance before deployment.
   - Few-Shot Episode Testing
     - Each test episode includes N classes and k samples per class (support set).
     - Accuracy is averaged over multiple episodes to ensure robust evaluation
2. Web Application Testing
   - Functional Testing : Ensures each feature of the Flask app works as expected.
   - Usability Testing:
     - Verifies that the UI is user-friendly and intuitive.
     - Test with potential users (e.g., students or faculty) to gather feedback.
   - Performance Testing
     - Measures response time of predictions
     - Ensures the app can handle multiple users (if applicable).
   - Edge Cases Testing
     - Tests unusual cases like:
       * Blurry or dark images
       * Images of non-plant objects
       * Extremely large or small files
   - Error Handling Tests
     - Ensures the system displays meaningful error messages:
       * If no image is uploaded
       * If the file type is not supported
       * If the model prediction fails

7.2 Test cases

No.	Functional Test Cases	Actual Output	Expected Output	Test Status
1	User Opens home page.	Home page loads with title and buttons.	Home page should load without errors.	Pass
2	User signs up with new credentials.	Account is created, user redirected.	New user should be registered and logged in.	Pass
3	User tries to sign up with existing email.	Error message shown.	Email already exists message shown	Pass
4	User logs in with correct credentials	Redirects to dashboard/upload page.	User should be authenticated and redirected.	Pass
5	User logs in with incorrect credentials	Error message shown	"Invalid credentials" message shown.	Pass

Table 7.1: Functional Testing Test Cases

No.	Integration Test Cases	Actual Output	Expected Output	Test Status
1	User signs up and gets redirected to login.	Signup → Database → Redirect	Redirects to login page after storing user data.	Pass
2	User logs in with valid credentials.	Login → Database → Dashboard	User is taken to dashboard	Pass
3	Logged-in user uploads image for prediction.	Upload → Model → Result Display.	Displays plant type and disease name	Pass
4	User uploads a valid plant leaf image.	Disease name and plant name displayed	Model should predict and show results.	Pass

Table 7.2: Integration Testing Test Cases

No.	System Test Cases	Actual Output	Expected Output	Test Status
1	User signs up and logs in.	Redirect to dashboard after login.	Redirects successfully.	Pass
2	Upload valid plant leaf image.	Leaf image (e.g., tomato$_b light.jpg$).	Output correct	Pass
3	Upload non-leaf image.	Shows "Invalid image" or "No plant detected.	Displays error as expected.	Pass
4	Logout functionality.	Redirects to login, session ends.	Redirects and session clears.	Pass

Table 7.3: System Testing Test Cases

CHAPTER 8

RESULTS

8.1 Outcomes

The developed system was evaluated using multiple testing methods, including unit testing, integration testing, and system testing. The goal was to ensure that the application functions reliably across all its components, from image upload and disease prediction to user authentication.

8.1.1 Functional Accuracy

The few-shot learning model trained with the hierarchical prototypical network achieved high accuracy even with limited data. Training Accuracy: 92.6% Validation Accuracy: 95.73%

8.2 Testing Results

1. Unit testing summary
   - All major individual components, such as image upload, prediction engine, and user authentication, were tested independently and passed successfully.
2. Integration Testing Summary
   - Integrated modules (e.g., login + prediction page access, upload + model inference) worked together seamlessly. Inter-module communication showed no significant errors or lags
3. System Testing Summary

No.	Test Scenario Description	Status
1	User signup and login flow	Pass
2	Upload a valid plant leaf image and predict disease	Pass
3	Upload an invalid image (non-leaf)	Pass
4	Access prediction page without login	Pass
5	Logout and session termination	Pass

Table 8.1: System Testing Summary

8.3 Observations

- The model performed robustly across unseen samples.
- The user interface was responsive and handled errors gracefully (e.g., unsupported image formats).
- The backend Flask server efficiently managed multiple prediction requests without crashing.

8.4 Interface Snapshots

To demonstrate the working of the Flask-based web application, the following screenshots are included:

- Home Page Interface

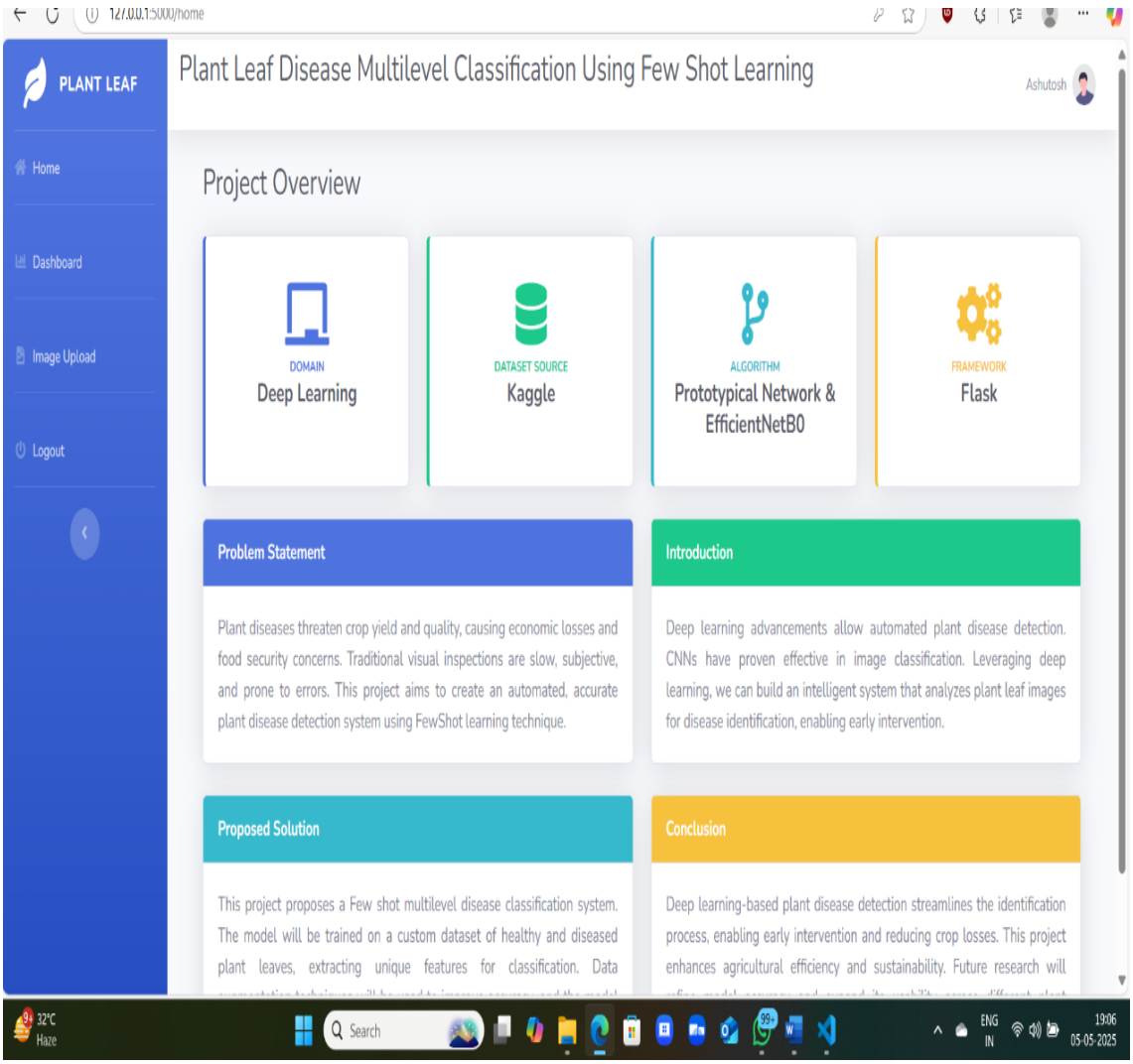

Figure 8.1: Home Page Interface

- User Login Page

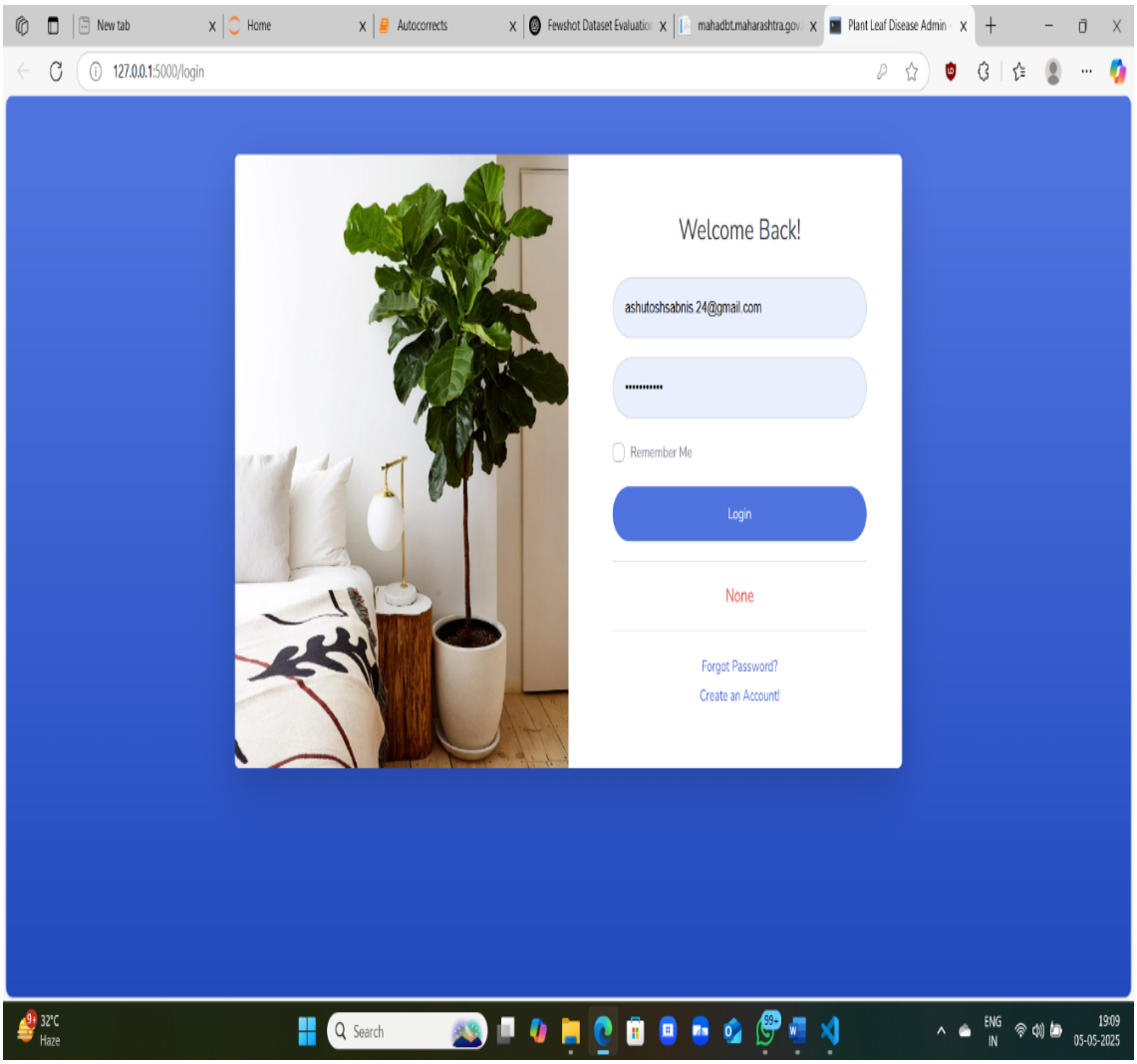

Figure 8.2: User Login Page

- Dashboard

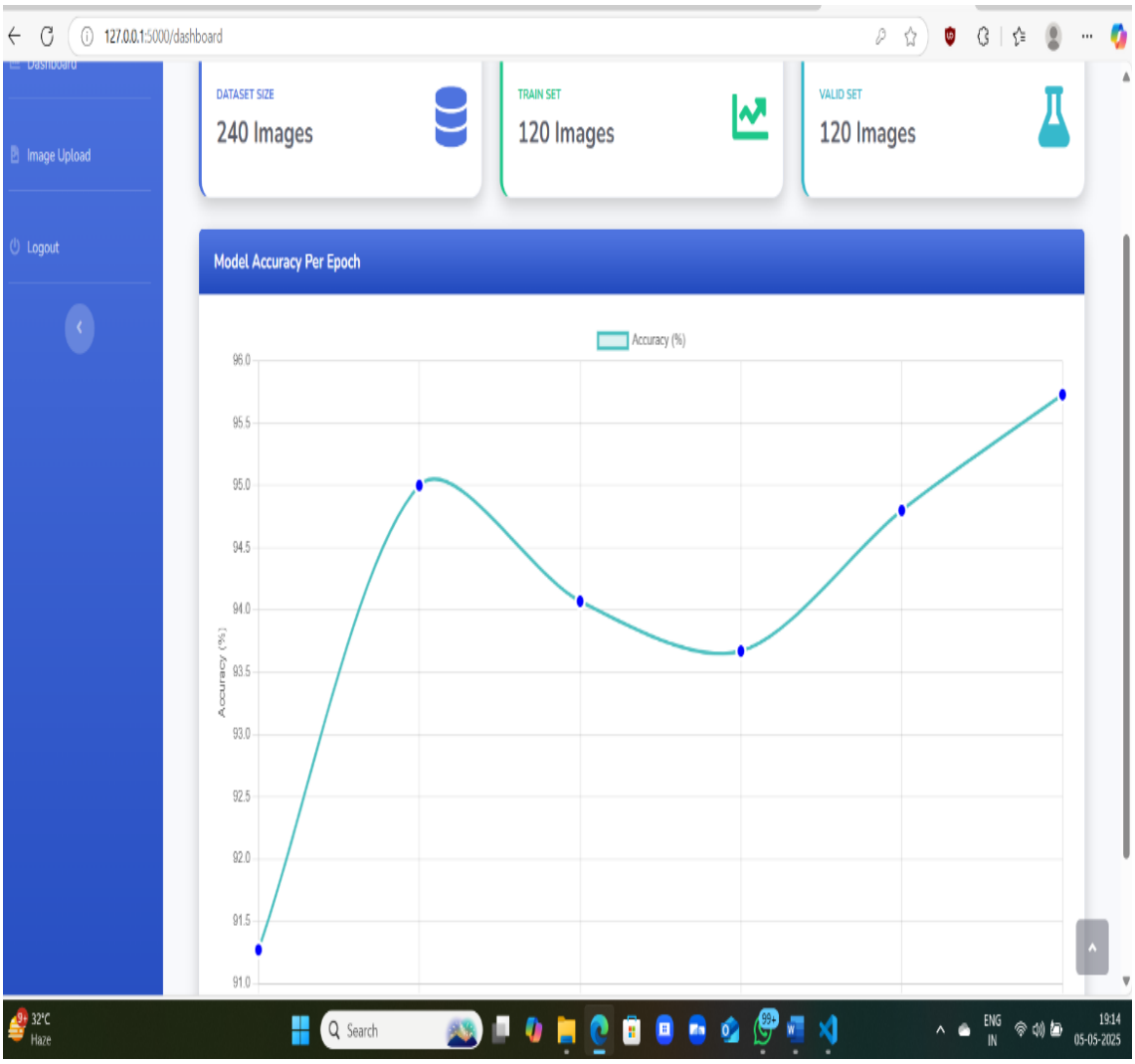

Figure 8.3: Dashboard

- Prediction Output Page

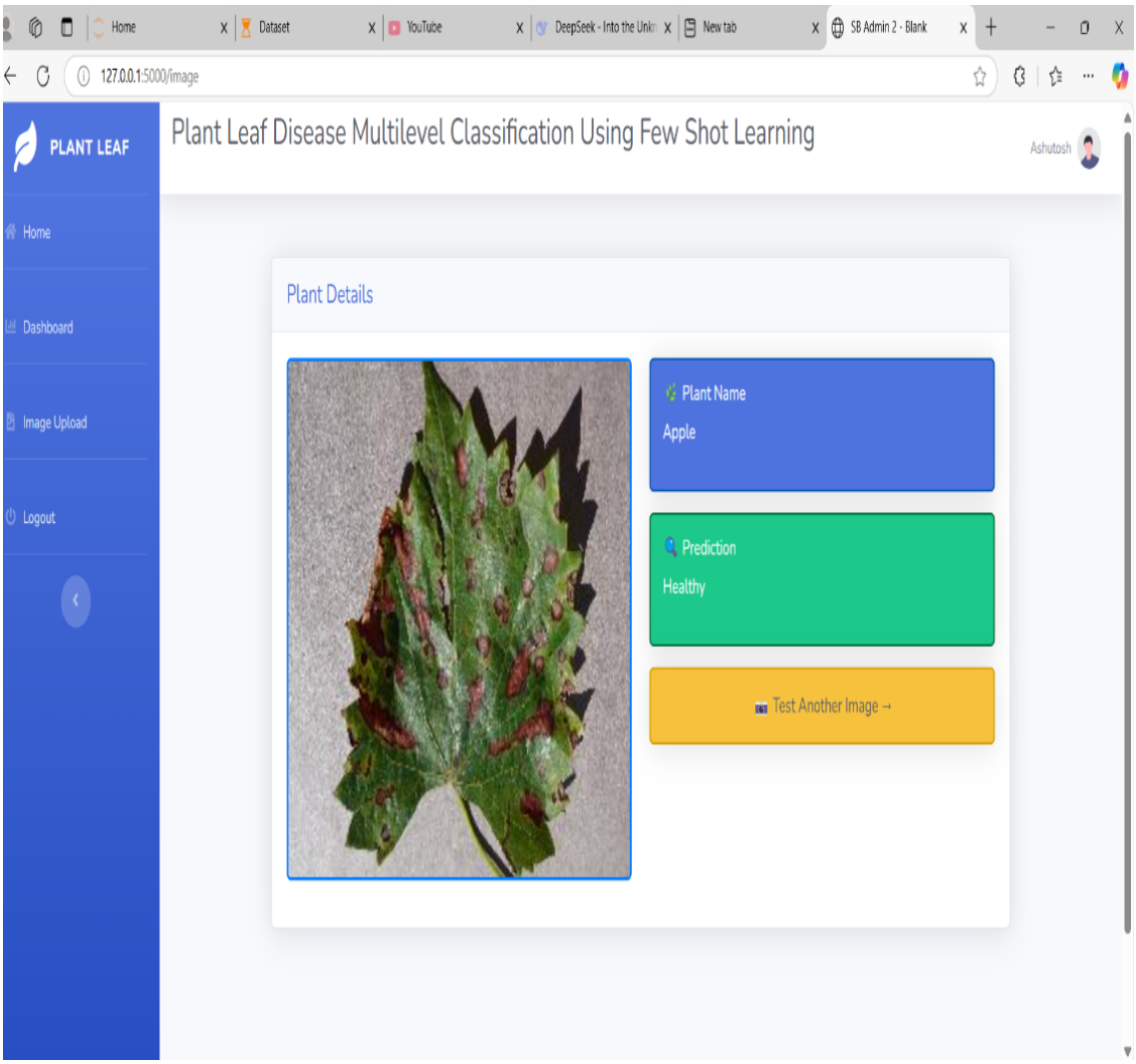

Figure 8.4: Prediction Output Page

CHAPTER 9
CONCLUSIONS

9.1 Conclusions

The proposed system successfully implements a plant leaf classification and disease detection model using a few-shot learning approach with a hierarchical prototypical network and EfficientNet-B0 backbone. This method provides accurate predictions even with limited training samples, making it highly efficient and practical for real-world agricultural applications. The integration of this model with a Flask-based web application further demonstrates its usability, allowing users to upload plant leaf images and receive instant classification and diagnosis results. The system is lightweight, scalable, and adaptable to new plant types and diseases with minimal additional data.

9.2 Future Scope

1. Model Extension: Future work can include extending the model to support more plant species and rare diseases by incorporating external agricultural datasets.
2. Mobile App Deployment: The system can be deployed as a mobile application for farmers to access predictions offline using edge devices..
3. Real-time Detection: Integration with live camera feeds from drones or mobile devices for real-time disease monitoring.

9.3 Applications

1. Agricultural Diagnosis Tools: Used by farmers or agricultural officers for early diagnosis of plant diseases to prevent crop loss.
2. Educational Tool : Demonstrates machine learning applications in botany and agricultural science.
3. Precision Farming: Helps improve yield by enabling timely interventions through accurate plant health monitoring.
4. Research: Useful for botanical research, dataset generation, and testing novel few-shot learning algorithms.

9.4 Limitations

1. The current system is trained on the PlantVillage dataset, which may not fully represent field conditions such as complex backgrounds or poor lighting.
2. Requires good image quality for accurate predictions.
3. The few-shot model still needs some samples per class to function—performance degrades for entirely unseen categories without support samples.
4. The web application does not currently include user analytics, image correction, or geo location tagging.

ANNEXURE A

1. Few-Shot Learning (FSL) :
   - Few-shot learning is a branch of machine learning where models are trained to classify data using only a few labeled examples per class. It is especially useful when acquiring large amounts of labeled data is impractical. In this project, a Prototypical Network is used for FSL, which computes class prototypes and compares query samples using a distance metric.
2. Prototypical Networks :
   - Prototypical Networks are a type of metric-based learning model used for few-shot classification. Each class is represented by the mean (prototype) of the embeddings of its support samples. During inference, a query sample is classified based on its distance to these prototypes in the embedding space.
3. EfficientNet-B0 :
   - EfficientNet is a family of Convolutional neural networks that scales depth, width, and resolution in a balanced way. The B0 version is the baseline model, offering a good trade-off between accuracy and efficiency. It is used as the backbone for feature extraction in this project. .
4. Episodic Training:
   - This technique simulates the few-shot setting during training. Each episode contains a small support set and a query set, mimicking the real inference conditions. It helps the model generalize better to unseen classes.
5. Flask Web Application
   - Flask is a lightweight Python web framework used to build the frontend and backend of the web application. It allows users to upload leaf images, performs model inference, and returns the predicted plant and disease name
6. Dataset Cleaning
   - To improve performance and reduce noise, images with low cosine similarity to class prototypes (less than a set threshold) are removed during the cleaning phase. This ensures only relevant examples are used for training.
7. Web App Functionalities
   - User Authentication: Signup and Login functionality using Flask sessions.
   - Image Upload: Allows users to upload plant leaf images.
   - Prediction: Returns both the plant type and disease class.

ANNEXURE B

- The complete source code for the project, including data preprocessing, few-shot model training using a Prototypical Network with EfficientNet-B0 as the backbone, and the Flask web application for user interaction, has been included in this Annexure. It contains modules for data loading, episodic training, model evaluation, image uploading, and prediction display, as well as the login and Signup system. All scripts are well-documented for better understanding and future reference.

ANNEXURE C

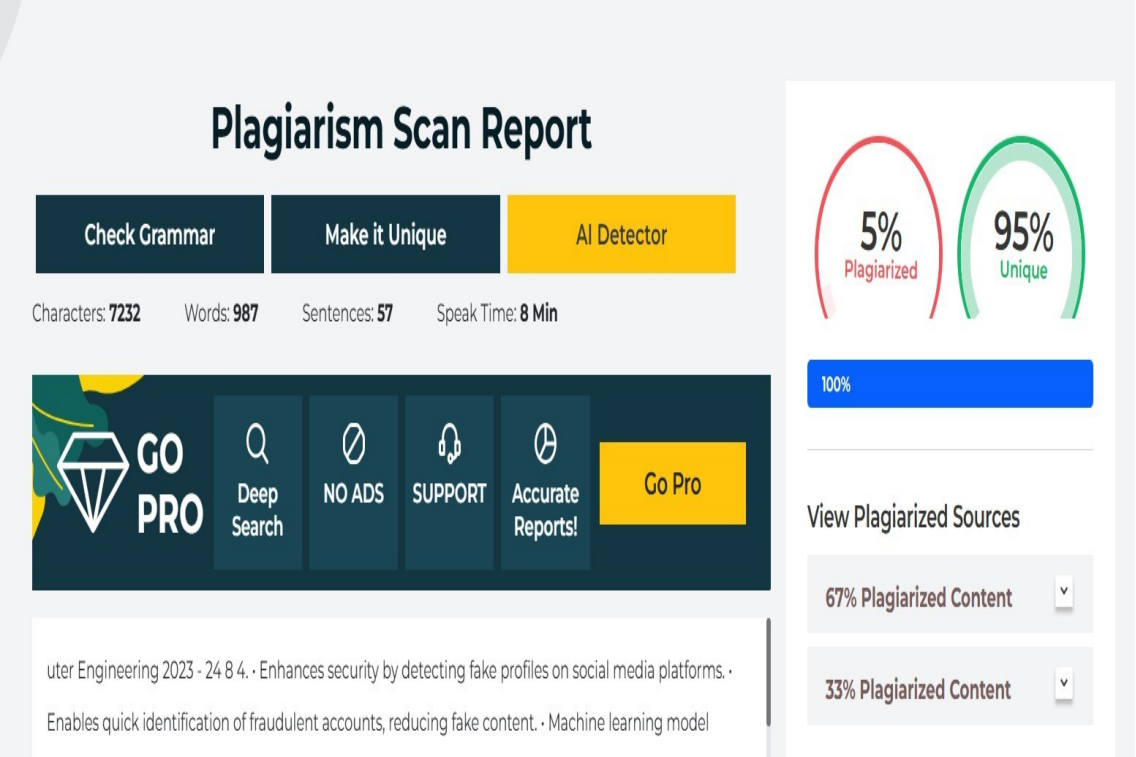

Figure C.1: Plagiarism Report

CHAPTER 10

REFERENCES

[1] Bhargava, Anuja, Aasheesh Shukla, Om Goswami, Mohammed H. Alsharif, Peerapong Uthansakul, and Monthippa Uthansakul. "Plant Leaf Disease Detection, Classification and Diagnosis using Computer Vision and Artificial Intelligence: A Review." *IEEE Access*, 2024.

[2] Li, Lili, Shujuan Zhang, and Bin Wang. "Plant disease detection and classification by deep learning—a review." *IEEE Access*, vol. 9, pp. 56683–56698, 2021.

[3] Singh, R. Satya Rajendra, and Rakesh Kumar Sanodiya. "Zero-Shot Transfer Learning Framework for Plant Leaf Disease Classification." *IEEE Access*, 2023.

[4] Kumar, Bishnu Deo, Himanshu Pandey, Nikhil Prakash, and Prajjwal Tripathi. "One-Shot Learning Methodology for Plant Leaf Disease Identification." In *2023 3rd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE)*, pp. 719–724. IEEE, 2023.

[5] Balafas, Vasileios, Emmanouil Karantoumanis, Malamati Louta, and Nikolaos Ploskas. "Machine Learning and Deep Learning for Plant Disease Classification and Detection." *IEEE Access*, 2023.

[6] Sharma, Pushkara, Pankaj Hans, and Subhash Chand Gupta. "Classification of Plant Leaf Diseases using Machine Learning and Image Preprocessing Techniques." In *2020 10th International Conference on Cloud Computing, Data Science and Engineering (Confluence)*, pp. 480–484. IEEE, 2020.

[7] Kumar, Sandeep, K. M. V. V. Prasad, A. Srilekha, T. Suman, B. Pranav Rao, and J. Naga Vamshi Krishna. "Leaf Disease Detection and Classification Based on Machine Learning." In *2020 International Conference on Smart Technologies in Computing, Electrical and Electronics (ICSTCEE)*, pp. 361–365. IEEE, 2020.